Unet

U-Net is a convolutional neural network architecture mainly used for **image segmentation**, especially in biomedical image analysis. It was introduced in 2015 by Olaf Ronneberger and his team at University of Freiburg. The architecture gets its name from its **U-shaped structure**, which consists of two main parts: a **contracting path (encoder)** that captures context by reducing spatial dimensions, and an **expanding path (decoder)** that restores spatial resolution for precise localization. A key feature of U-Net is its **skip connections**, which directly connect corresponding encoder and decoder layers to preserve fine-grained details. Because of this design, U-Net works very well even with limited training data and is widely used in medical imaging, satellite image segmentation, and object detection tasks.

```
import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf
from tensorflow.keras import layers, models
# Create dummy grayscale images (128x128)
num_samples = 200
img_size = 128

X = np.random.rand(num_samples, img_size, img_size, 1)

# Create dummy circular infection masks
Y = np.zeros_like(X)

for i in range(num_samples):
    center_x = np.random.randint(40, 90)
    center_y = np.random.randint(40, 90)
    radius = np.random.randint(10, 25)
```

```
    for x in range(img_size):
        for y in range(img_size):
            if (x - center_x)**2 + (y - center_y)**2 < radius**2:
                Y[i, x, y, 0] = 1
def build_unet(input_size=(128,128,1)):

    inputs = layers.Input(input_size)

    # Encoder
    c1 = layers.Conv2D(32, 3, activation='relu', padding='same')(inputs)
    c1 = layers.Conv2D(32, 3, activation='relu', padding='same')(c1)
    p1 = layers.MaxPooling2D((2,2))(c1)

    c2 = layers.Conv2D(64, 3, activation='relu', padding='same')(p1)
    c2 = layers.Conv2D(64, 3, activation='relu', padding='same')(c2)
    p2 = layers.MaxPooling2D((2,2))(c2)

    # Bottleneck
    c3 = layers.Conv2D(128, 3, activation='relu', padding='same')(p2)
    c3 = layers.Conv2D(128, 3, activation='relu', padding='same')(c3)

    # Decoder
    u1 = layers.UpSampling2D((2,2))(c3)
    u1 = layers.concatenate([u1, c2])
    c4 = layers.Conv2D(64, 3, activation='relu', padding='same')(u1)
    c4 = layers.Conv2D(64, 3, activation='relu', padding='same')(c4)

    u2 = layers.UpSampling2D((2,2))(c4)
    u2 = layers.concatenate([u2, c1])
    c5 = layers.Conv2D(32, 3, activation='relu', padding='same')(u2)
    c5 = layers.Conv2D(32, 3, activation='relu', padding='same')(c5)

    outputs = layers.Conv2D(1, 1, activation='sigmoid')(c5)

    model = models.Model(inputs, outputs)
    return model
model = build_unet()
model.compile(optimizer='adam',
              loss='binary_crossentropy',
              metrics=['accuracy'])
```

```
model.summary()
```

Layer (type)	Output Shape	Param #	Connected to
input_layer (InputLayer)	(None, 128, 128, 1)	0	-
conv2d (Conv2D)	(None, 128, 128, 32)	320	input_layer[0][0]
conv2d_1 (Conv2D)	(None, 128, 128, 32)	9,248	conv2d[0][0]
max_pooling2d (MaxPooling2D)	(None, 64, 64, 32)	0	conv2d_1[0][0]
conv2d_2 (Conv2D)	(None, 64, 64, 64)	18,496	max_pooling2d[0]…
conv2d_3 (Conv2D)	(None, 64, 64, 64)	36,928	conv2d_2[0][0]
max_pooling2d_1 (MaxPooling2D)	(None, 32, 32, 64)	0	conv2d_3[0][0]
conv2d_4 (Conv2D)	(None, 32, 32, 128)	73,856	max_pooling2d_1[…
conv2d_5 (Conv2D)	(None, 32, 32, 128)	147,584	conv2d_4[0][0]
up_sampling2d (UpSampling2D)	(None, 64, 64, 128)	0	conv2d_5[0][0]
concatenate (Concatenate)	(None, 64, 64, 192)	0	up_sampling2d[0]… conv2d_3[0][0]

conv2d_5 (Conv2D)	(None, 32, 32, 128)	147,584	conv2d_4[0][0]
up_sampling2d (UpSampling2D)	(None, 64, 64, 128)	0	conv2d_5[0][0]
concatenate (Concatenate)	(None, 64, 64, 192)	0	up_sampling2d[0]… conv2d_3[0][0]
conv2d_6 (Conv2D)	(None, 64, 64, 64)	110,656	concatenate[0][0]
conv2d_7 (Conv2D)	(None, 64, 64, 64)	36,928	conv2d_6[0][0]
up_sampling2d_1 (UpSampling2D)	(None, 128, 128, 64)	0	conv2d_7[0][0]
concatenate_1 (Concatenate)	(None, 128, 128, 96)	0	up_sampling2d_1[… conv2d_1[0][0]
conv2d_8 (Conv2D)	(None, 128, 128, 32)	27,680	concatenate_1[0]…
conv2d_9 (Conv2D)	(None, 128, 128, 32)	9,248	conv2d_8[0][0]
conv2d_10 (Conv2D)	(None, 128, 128, 1)	33	conv2d_9[0][0]

Total params: 470,977 (1.80 MB)
Trainable params: 470,977 (1.80 MB)
Non-trainable params: 0 (0.00 B)

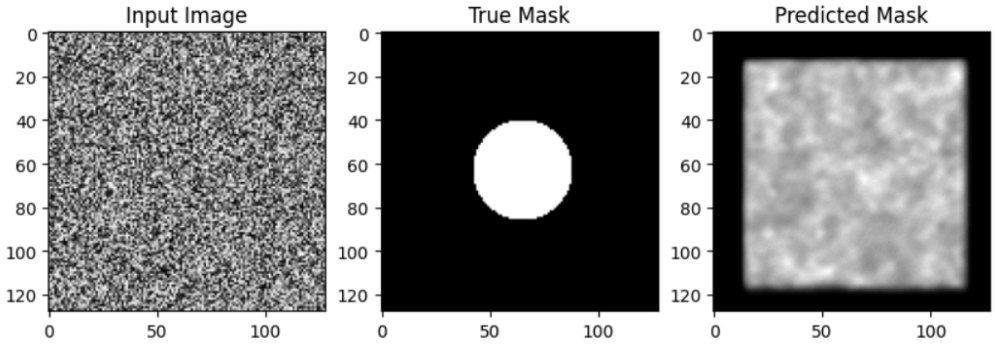
```
history = model.fit(X, Y,
                    validation_split=0.2,
                    epochs=5,
                    batch_size=8)
```

```
Epoch 1/5
20/20 ━━━━━━━━━━━━━━━━━━━━ 64s 3s/step - accuracy: 0.8611 - loss: 0.4403 - val_accuracy: 0.9470 - val_loss: 0.2248
Epoch 2/5
20/20 ━━━━━━━━━━━━━━━━━━━━ 59s 3s/step - accuracy: 0.9443 - loss: 0.2311 - val_accuracy: 0.9470 - val_loss: 0.2106
Epoch 3/5
20/20 ━━━━━━━━━━━━━━━━━━━━ 82s 3s/step - accuracy: 0.9389 - loss: 0.2237 - val_accuracy: 0.9470 - val_loss: 0.1969
Epoch 4/5
20/20 ━━━━━━━━━━━━━━━━━━━━ 59s 3s/step - accuracy: 0.9433 - loss: 0.2038 - val_accuracy: 0.9470 - val_loss: 0.1832
Epoch 5/5
20/20 ━━━━━━━━━━━━━━━━━━━━ 57s 3s/step - accuracy: 0.9398 - loss: 0.2151 - val_accuracy: 0.9470 - val_loss: 0.1893
```

```
pred = model.predict(X[:1])

plt.figure(figsize=(10,4))

plt.subplot(1,3,1)
plt.title("Input Image")
```

```
plt.imshow(X[0].squeeze(), cmap='gray')

plt.subplot(1,3,2)
plt.title("True Mask")
plt.imshow(Y[0].squeeze(), cmap='gray')

plt.subplot(1,3,3)
plt.title("Predicted Mask")
plt.imshow(pred[0].squeeze(), cmap='gray')

plt.show()
```

```
1/1 ━━━━━━━━━━━━━━━━━━━━ 3s 3s/step
```




```
import gradio as gr
import time

# ==================================================
# 1️⃣ System Prompt (Conceptual - for architecture)
# ==================================================
SYSTEM_PROMPT = """
You are a clinical medical assistant specialized in diabetes management.
Answer using retrieved structured knowledge only.
"""

# ==================================================
# 2️⃣ Hierarchical Medical Corpus
# ==================================================
medical_corpus = {
    "Diabetes": {
        "Symptoms": [
            "Increased thirst, frequent urination, fatigue."
        ],
```

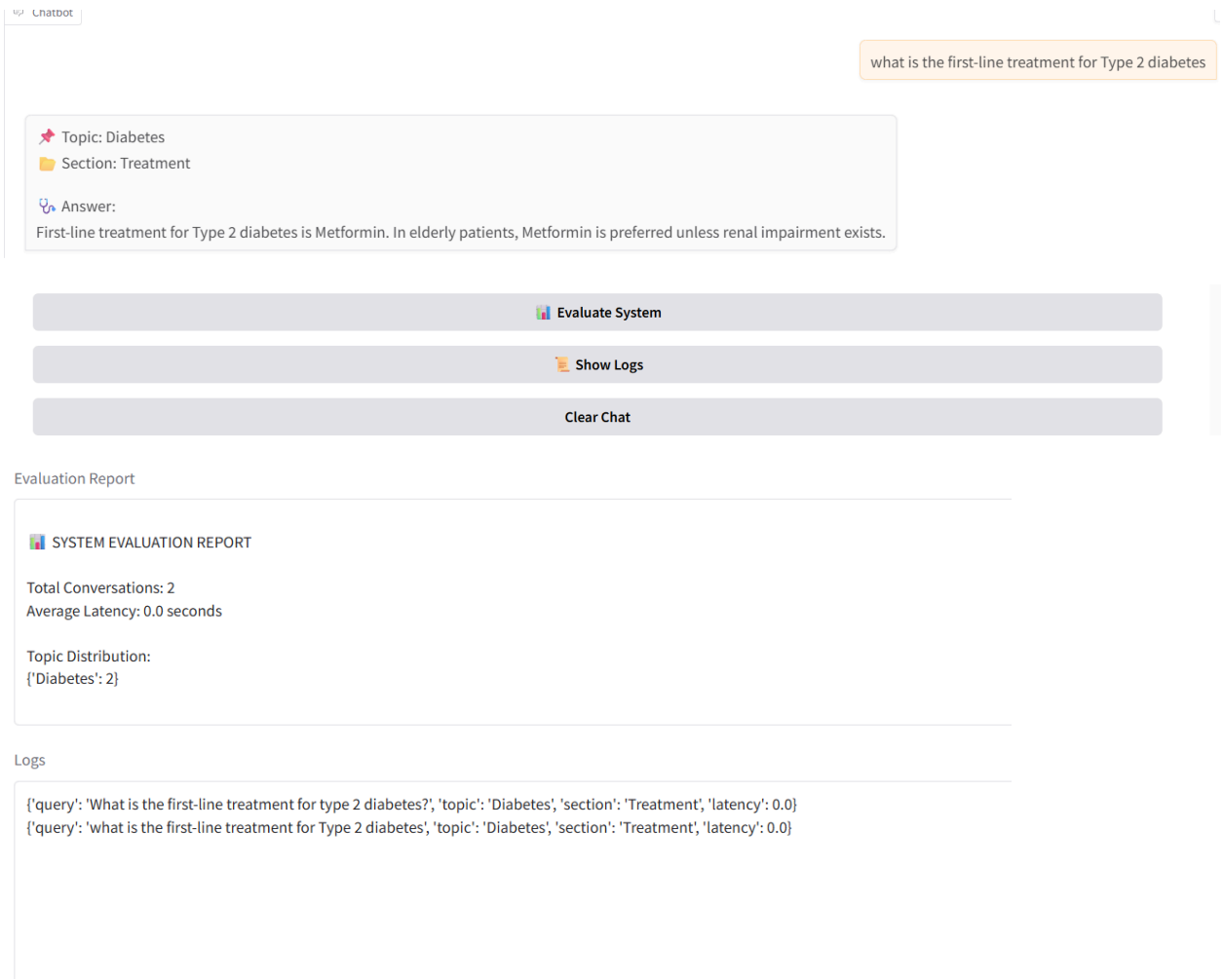
```
        "Treatment": [
            "First-line treatment for Type 2 diabetes is Metformin.",
            "In elderly patients, Metformin is preferred unless renal
impairment exists."
        ],
        "Complications": [
            "Neuropathy, nephropathy, retinopathy."
        ]
    },
    "Hypertension": {
        "Treatment": [
            "First-line therapy includes ACE inhibitors."
        ]
    }
}

# ====================================================
# 3️⃣ Memory & Logs
# ====================================================
conversation_memory = []
logs = []

# ====================================================
# 4️⃣ Hierarchical Retrieval
# ====================================================
def hierarchical_retrieval(query, corpus):

    topic = None
    for key in corpus.keys():
        if key.lower() in query.lower():
            topic = key
            break

    if topic is None:
        topic = "Diabetes"  # fallback

    section = None
    for sec in corpus[topic].keys():
        if sec.lower() in query.lower():
            section = sec
```

```
            break

    if section is None:
        section = "Treatment"

    context = " ".join(corpus[topic][section])

    return topic, section, context

# ====================================================
# 5️⃣ Medical Agent (Orchestration + Observability)
# ====================================================
def medical_agent(query):

    start_time = time.time()

    topic, section, context = hierarchical_retrieval(query,
medical_corpus)

    answer = f"""
📌 Topic: {topic}
📂 Section: {section}

🩺 Answer:
{context}
"""

    latency = round(time.time() - start_time, 4)

    # Store conversation
    conversation_memory.append({
        "query": query,
        "response": answer
    })

    # Store logs (OBSERVABILITY)
    logs.append({
        "query": query,
        "topic": topic,
        "section": section,
```

```
        "latency": latency
    })

    return answer

# ==================================================
# 6️⃣ Chat Interface
# ==================================================
def chat_interface(message, history):

    if history is None:
        history = []

    response = medical_agent(message)

    history.append({"role": "user", "content": message})
    history.append({"role": "assistant", "content": response})

    return history, ""

# ==================================================
# 7️⃣ Evaluation (Observability Metrics)
# ==================================================
def evaluate():

    total_conversations = len(conversation_memory)

    if len(logs) == 0:
        return "No data available."

    avg_latency = sum(log["latency"] for log in logs) / len(logs)

    topic_distribution = {}
    for log in logs:
        topic = log["topic"]
        topic_distribution[topic] = topic_distribution.get(topic, 0) + 1

    report = f"""
📊 SYSTEM EVALUATION REPORT
```

```
Total Conversations: {total_conversations}
Average Latency: {round(avg_latency,4)} seconds

Topic Distribution:
{topic_distribution}
"""

    return report

# ================================================
# 8️⃣ Log Viewer
# ================================================
def show_logs():
    if not logs:
        return "No logs yet."

    return "\n".join(str(log) for log in logs)

# ================================================
# 9️⃣ Gradio UI
# ================================================
with gr.Blocks() as demo:

    gr.Markdown("# 🩺 Hierarchical RAG Medical Assistant")
    gr.Markdown("Includes Observability & Evaluation (Step 7)")

    chatbot = gr.Chatbot(type="messages")
    msg = gr.Textbox(placeholder="Ask Medical Assistant...")

    eval_btn = gr.Button("📊 Evaluate System")
    log_btn = gr.Button("📜 Show Logs")
    clear_btn = gr.Button("Clear Chat")

    eval_output = gr.Textbox(label="Evaluation Report", lines=8)
    log_output = gr.Textbox(label="Logs", lines=10)

    msg.submit(
        chat_interface,
        inputs=[msg, chatbot],
        outputs=[chatbot, msg]
```

```
    )

    eval_btn.click(
        evaluate,
        inputs=None,
        outputs=eval_output
    )

    log_btn.click(
        show_logs,
        inputs=None,
        outputs=log_output
    )
    clear_btn.click(lambda: [], None, chatbot)

demo.launch()
```

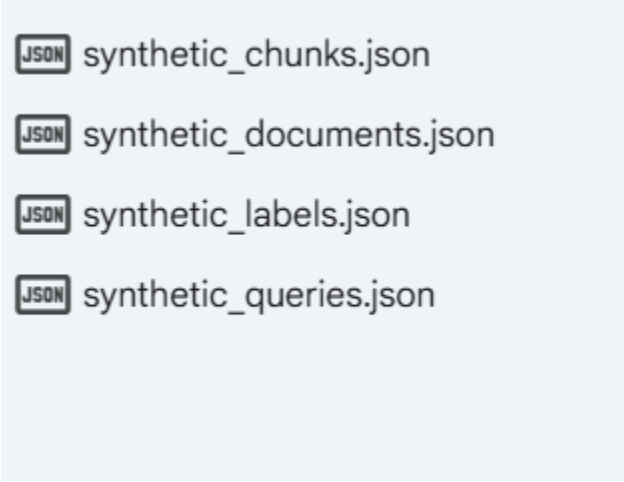


```
import random
import uuid
import json

domains = {
    "Artificial Intelligence": [
        "Neural networks consist of multiple layers including input, hidden, and output layers.",
        "Transformer models use self-attention mechanisms to capture contextual relationships.",
        "Reinforcement learning agents optimize policies using reward feedback.",
        "Large Language Models are trained on massive corpora using next-token prediction."
    ],
    "Cybersecurity": [
        "Encryption ensures confidentiality using symmetric and asymmetric algorithms.",
        "Public key infrastructure enables secure communication over insecure channels.",
        "Firewalls monitor incoming and outgoing traffic based on security rules.",
        "Zero trust architecture assumes no implicit trust within networks."
    ],
    "Healthcare": [
        "Machine learning models assist in early disease detection.",
        "Electronic health records store patient medical histories digitally.",
        "Telemedicine enables remote consultations between doctors and patients.",
        "Vaccines stimulate immune responses to prevent infections."
    ]
}

def generate_document(doc_id):
    domain = random.choice(list(domains.keys()))
    content = []
```

```
    for section_id in range(5):
        section_title = f"Section {section_id+1}: {domain} Topic
{section_id+1}"
        paragraphs = random.sample(domains[domain], 3)
        content.append({
            "section_title": section_title,
            "paragraphs": paragraphs
        })

    return {
        "doc_id": doc_id,
        "domain": domain,
        "content": content
    }

documents = [generate_document(str(uuid.uuid4())) for _ in range(50)]

with open("synthetic_documents.json", "w") as f:
    json.dump(documents, f, indent=4)

def adaptive_chunk(document):
    chunks = []

    for section in document["content"]:
        # Section-level chunk
        section_text = " ".join(section["paragraphs"])
        chunks.append({
            "chunk_id": str(uuid.uuid4()),
            "doc_id": document["doc_id"],
            "chunk_type": "section",
            "text": section_text
        })

        # Paragraph-level chunks
        for para in section["paragraphs"]:
            chunks.append({
                "chunk_id": str(uuid.uuid4()),
                "doc_id": document["doc_id"],
                "chunk_type": "paragraph",
                "text": para
```

```
            })

    return chunks

all_chunks = []
for doc in documents:
    all_chunks.extend(adaptive_chunk(doc))

with open("synthetic_chunks.json", "w") as f:
    json.dump(all_chunks, f, indent=4)
query_templates = [
    "Explain how {concept} works.",
    "What is the role of {concept} in {domain}?",
    "How does {concept} improve system performance?",
    "Describe the importance of {concept}."
]

def generate_queries(documents, num_queries=100):
    queries = []

    for _ in range(num_queries):
        doc = random.choice(documents)
        section = random.choice(doc["content"])
        concept = random.choice(section["paragraphs"]).split()[0]

        query = random.choice(query_templates).format(
            concept=concept,
            domain=doc["domain"]
        )

        queries.append({
            "query_id": str(uuid.uuid4()),
            "query": query,
            "domain": doc["domain"]
        })

    return queries

queries = generate_queries(documents)
```

```
with open("synthetic_queries.json", "w") as f:
    json.dump(queries, f, indent=4)
def generate_ground_truth(queries, chunks):
    ground_truth = []

    for q in queries:
        relevant_chunks = []
        keyword = q["query"].split()[2]  # crude keyword extraction

        for chunk in chunks:
            if keyword.lower() in chunk["text"].lower():
                relevant_chunks.append(chunk["chunk_id"])

        ground_truth.append({
            "query_id": q["query_id"],
            "relevant_chunks": relevant_chunks[:5]
        })

    return ground_truth

labels = generate_ground_truth(queries, all_chunks)

with open("synthetic_labels.json", "w") as f:
    json.dump(labels, f, indent=4)
```




```
# ========================================
# Stable Diffusion + Classical Denoising
# ========================================
```

```python
import numpy as np
import matplotlib.pyplot as plt
import cv2

# ----------------------------------------
# 1. Create Synthetic Image
# ----------------------------------------
img = np.zeros((128, 128), dtype=np.float32)
cv2.putText(img, '5', (35, 95), cv2.FONT_HERSHEY_SIMPLEX, 3, (1), 5)
img = img / img.max()

# ----------------------------------------
# 2. Forward Diffusion Process
# ----------------------------------------
def forward_diffusion(x0, alpha):
    noise = np.random.normal(0, 1, x0.shape)
    xt = np.sqrt(alpha) * x0 + np.sqrt(1 - alpha) * noise
    return xt, noise

alpha_values = [0.8, 0.5, 0.2]
noisy_images = []

for alpha in alpha_values:
    xt, noise = forward_diffusion(img, alpha)
    noisy_images.append(xt)

# ----------------------------------------
# 3. Reverse Diffusion (Simulated)
# ----------------------------------------
alpha = 0.2
xt, true_noise = forward_diffusion(img, alpha)
recovered = (xt - np.sqrt(1 - alpha) * true_noise) / np.sqrt(alpha)

# ----------------------------------------
# 4. Classical Denoising
# ----------------------------------------

# Add Gaussian noise
noise = np.random.normal(0, 0.5, img.shape)
noisy_img = img + noise
```

```python
# Mean Filter
mean_filtered = cv2.blur(noisy_img, (5,5))

# Median Filter
median_filtered = cv2.medianBlur((noisy_img*255).astype(np.uint8), 5)
median_filtered = median_filtered.astype(np.float32)/255

# Gaussian Filter
gaussian_filtered = cv2.GaussianBlur(noisy_img, (5,5), 0)

# ----------------------------------------
# 5. MSE Function
# ----------------------------------------
def mse(original, denoised):
    return np.mean((original - denoised) ** 2)

print("MSE Mean Filter:", mse(img, mean_filtered))
print("MSE Median Filter:", mse(img, median_filtered))
print("MSE Gaussian Filter:", mse(img, gaussian_filtered))

# ----------------------------------------
# 6. Display Results
# ----------------------------------------
titles = [
    "Original",
    "Forward Diffusion (alpha=0.2)",
    "Reverse Diffusion",
    "Noisy Image",
    "Mean Filter",
    "Median Filter",
    "Gaussian Filter"
]

images = [
    img,
    xt,
    recovered,
    noisy_img,
    mean_filtered,
```

```
    median_filtered,
    gaussian_filtered
]

for i in range(len(images)):
    plt.figure()
    plt.title(titles[i])
    plt.imshow(images[i], cmap='gray')
    plt.axis("off")
    plt.show()

print("Done ✅")
```

```
MSE Mean Filter: 0.01931256652865898
MSE Median Filter: 0.26088738
MSE Gaussian Filter: 0.024295516579573267
```

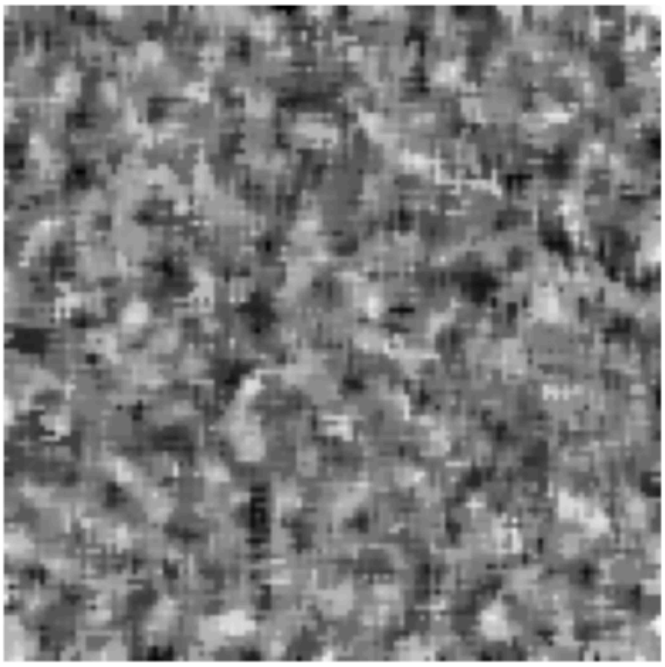
Forward Diffusion (alpha=0.2)

Reverse Diffusion

Noisy Image

Mean Filter

Median Filter



Gaussian Filter

Done ✅